Dataflow Architectures. Design of AXI4-Stream Compliant Modules

1 Purpose

This laboratory work presents the internal behaviour of AXI4-Stream compliant hardware modules. The main objective is to describe the mechanism used by AXI4-Stream modules to exchange data and to show how this mechanism can be implemented, in a simple and systematic way, when developing such hardware components.

2 AXI4-Stream Internal Behaviour

As described in the previous laboratory work, the I/O interfaces of AXI4-Stream modules must comply with the rules of the ready-valid handshake [1, 2]. A processing module is a receiver on each of its input interfaces and a sender on each of its output interfaces, so its implementation has to satisfy these rules on both sides.

In case of AXI4-Stream modules which perform simple operations in just one clock cycle, the internal behaviour can be synthetically represented as a two-state Finite State Machine (FSM). In one state, the module accepts data and performs the actual operation, while in the other state it provides the result at the output.

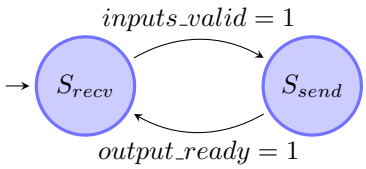


Figure 1: Internal behaviour of AXI4-Stream modules represented as a FSM

Figure 1 shows the state diagram which describes, in a simplified way, the operating mechanism of an AXI4-Stream module that is capable of performing the actual processing in a single clock cycle.

Table 1: FSM states

State	Description	Input TREADY	Output TVALID	Next state
S_{recv}	Waits until all inputs are valid, consumes them and performs the actual operation.	`inputs_valid`	LOW	S_{send}
S_{send}	Holds the result at the output until the receiver accepts it.	LOW	HIGH	S_{recv}

Table 1 shows the description of the two states of the FSM. Each state transition corresponds *exactly* to one handshake, so the transition conditions follow directly from the handshake definition presented in the previous laboratory work:

- $S_{recv} \rightarrow S_{send}$ is the handshake on the **input** interfaces. In S_{recv} the module drives every input **TREADY** with `inputs_valid` (the AND of all input **TVALID** signals), so the handshake condition reduces to `inputs_valid` $= 1$.
- $S_{send} \rightarrow S_{recv}$ is the handshake on the **output** interface. In S_{send} the output **TVALID** is always HIGH, so the handshake condition reduces to `output_ready` $= 1$ (the **TREADY** of the output interface).

Therefore, no additional condition (and no additional control signal) is needed to detect the handshakes.

The rules of the ready-valid handshake are the following:

Rule 1. Once **TVALID** is asserted it must remain asserted until the handshake occurs. This means that when the sender asserts **TVALID**, it must keep it HIGH until the receiver asserts **TREADY** and a rising edge of the **ACLK** appears.

Rule 2. A sender is not permitted to wait until **TREADY** is asserted before asserting **TVALID**. In other words, the sender must report when the data it provides is valid, even if the receiver is not ready to accept it.

Rule 3. A receiver is permitted to wait for **TVALID** to be asserted before asserting **TREADY**. This means that a receiver can wait for the input data to be valid before reporting that it is ready to accept it.

Rule 4. If a receiver asserts **TREADY**, it is permitted to deassert **TREADY** before **TVALID** is asserted. Thus, if the receiver sets the **TREADY** HIGH before the **TVALID** was asserted by the sender, it can set **TREADY** back to LOW and wait for the sender to report the data is valid (see **Rule 3**).

Table 2 shows how each of these rules is satisfied by this implementation.

Table 2: Compliance of the two-state implementation with the handshake rules

Rule	Interface	How the rule is ensured
Rule 1	output	The output **TVALID** is HIGH during the whole S_{send} state, which is left only after a handshake. The result register is written only in S_{recv}, so the output data does not change while waiting.
Rule 2	output	The output **TVALID** depends only on the current state, never on the output **TREADY**.
Rule 3	inputs	The input **TREADY** signals are asserted only when all input **TVALID** signals are HIGH. Because the senders obey **Rule 1**, an input that is valid earlier stays valid while the module waits for the others, so all inputs are consumed in the same clock cycle.
Rule 4	inputs	Never needed: the input **TREADY** signals are never asserted before **TVALID**, and once asserted the handshake occurs at the next rising edge.
Rule 4	output	The receiver may raise and lower **TREADY** while the output **TVALID** is LOW. The output **TREADY** is examined only in S_{send}, so such pulses are ignored.

If the input **TREADY** signals were asserted independently (e.g. HIGH during the whole S_{recv} state), an input that becomes valid earlier would be consumed before the others are available, and the module would have to store it and keep track of which inputs have already been received. Waiting for all **TVALID** signals (**Rule 3**) avoids this extra logic.

The simplified implementation spends at least one clock cycle in each state, so a new set of inputs is accepted at most once every two clock cycles.

3 Example of AXI4-Stream Compliant Module

This section provides an example of AXI4-Stream compliant adder/subtractor, implemented in VHDL using the approach described in the previous section. The meaning of the port names is the one explained in the previous laboratory work; `aresetn` is the active-LOW, synchronous reset of the AXI4-Stream interfaces.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_SIGNED.ALL;

entity int_adder_subtractor is
  Port (
    aclk : IN STD_LOGIC;
    aresetn : IN STD_LOGIC;
    s_axis_a_tvalid : IN STD_LOGIC;
    s_axis_a_tready : OUT STD_LOGIC;
    s_axis_a_tdata : IN STD_LOGIC_VECTOR(31 DOWNTO 0);
    s_axis_b_tvalid : IN STD_LOGIC;
    s_axis_b_tready : OUT STD_LOGIC;
    s_axis_b_tdata : IN STD_LOGIC_VECTOR(31 DOWNTO 0);
    s_axis_operation_tvalid : IN STD_LOGIC;
    s_axis_operation_tready : OUT STD_LOGIC;
    s_axis_operation_tdata : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
    m_axis_result_tvalid : OUT STD_LOGIC;
    m_axis_result_tready : IN STD_LOGIC;
    m_axis_result_tdata : OUT STD_LOGIC_VECTOR(31 DOWNTO 0)
  );
end int_adder_subtractor;

architecture Behavioral of int_adder_subtractor is

type state_type is (S_RECV, S_SEND);
signal state : state_type := S_RECV;

signal inputs_valid, inputs_ready : STD_LOGIC;
signal result : STD_LOGIC_VECTOR(31 downto 0) := (others => '0');

begin
    -- all input interfaces hold valid data
    inputs_valid <= s_axis_a_tvalid and s_axis_b_tvalid and s_axis_operation_tvalid;

    -- Rule 3: wait for all TVALIDs, so that all inputs are consumed together
    inputs_ready <= inputs_valid when state = S_RECV else '0';

    s_axis_a_tready <= inputs_ready;
    s_axis_b_tready <= inputs_ready;
    s_axis_operation_tready <= inputs_ready;

    -- Rules 1 and 2: TVALID depends only on the state
    m_axis_result_tvalid <= '1' when state = S_SEND else '0';
    m_axis_result_tdata <= result;

    process(aclk)
    begin
        if rising_edge(aclk) then
            if aresetn = '0' then
                state <= S_RECV;
```

```
            else
                case state is
                    when S_RECV =>
                        -- input handshake (TREADY = inputs_valid in this state)
                        if inputs_valid = '1' then
                            if s_axis_operation_tdata = x"00" then
                                result <= s_axis_a_tdata + s_axis_b_tdata;
                            else
                                result <= s_axis_a_tdata - s_axis_b_tdata;
                            end if;
                            state <= S_SEND;
                        end if;

                    when S_SEND =>
                        -- output handshake (TVALID = '1' in this state)
                        if m_axis_result_tready = '1' then
                            state <= S_RECV;
                        end if;
                end case;
            end if;
        end if;
    end process;

end Behavioral;
```

The performed operation is selected based on the `s_axis_operation_tdata` input, as described in Table 3. The operands are interpreted as signed (two's complement) integers, since the `STD_LOGIC_SIGNED` package defines the arithmetic operators directly on `STD_LOGIC_VECTOR` values.

Table 3: Operation selection

`s_axis_operation_tdata`	**Operation**
00000000	+
$\neq$ 00000000	-

When the value received on the input port `s_axis_operation_tdata` is equal to `00000000`, the module performs the addition of the values received on the input ports `s_axis_a_tdata` and `s_axis_b_tdata` and when the value of the `s_axis_operation_tdata` input signal is not equal to `00000000`, it performs the subtraction of the two values.

The internal signals are described in Table 4.

Table 4: Description of the internal signals

Signal	**Description**
`state`	Current state of the FSM
`inputs_valid`	Control signal which indicates that all inputs are valid
`inputs_ready`	**TREADY** driven on every input interface. Equal to `inputs_valid` in the `S_RECV` state and LOW in the `S_SEND` state, so all inputs are consumed at once
`result`	Register holding the result. It is written only when the inputs are consumed, so it remains stable while it is provided at the output

The output `m_axis_result_tvalid` and `m_axis_result_tdata` are driven only by registers (`state` and `result`), while the input **TREADY** signals are a combinational function of the input **TVALID** signals, as permitted by **Rule 3**. These considerations are synthetically described in Table 5.

Table 5: Handshake signals driven by the module

State	s_axis_*_tready	m_axis_result_tvalid
S_RECV	inputs_valid	LOW
S_SEND	LOW	HIGH

4 Exercises

1. **Implement an AXI4-Stream compliant module which saturates the value given as input so that it fits in a given range. The range limits are also provided as inputs. This module has to ensure that the output is in the given range by saturating the input value if necessary.**

   Design and implementation considerations:

   - Start from the adder shown in Section 3. The FSM and the handshake logic remain unchanged; only the input interfaces and the operation performed in the `S_RECV` state change.
   - The result is `max` if `val > max`, `min` if `val < min`, and `val` otherwise. All values are signed integers and it can be assumed that `min <= max`.

2. **Implement an AXI4-Stream compliant module which computes the sum of a window of values. The module gets a single value at once, then updates the internally stored window of values and provides at the output the sum of all the values in the window.**

   Design and implementation considerations:

   - Use the same two-state FSM, with a single input interface. The size of the window is given by the generic parameter `WINDOW_SIZE`, and the window, the pointer and the sum can be declared as follows:

```
type window_type is array (0 to WINDOW_SIZE - 1) of
                                        std_logic_vector(31 downto 0);
signal window : window_type := (others => (others => '0'));
signal ptr : integer range 0 to WINDOW_SIZE - 1 := 0;
signal sum : std_logic_vector(31 downto 0) := (others => '0');
```

   - The pointer always indicates the position of the *oldest* value in the window, which is replaced by the next input value. Figure 2 shows the content of the window after the values $x_0, \ldots, x_5$ have been read: the next input, x_6, will replace x_1.

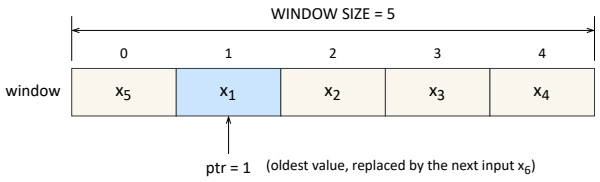


Figure 2: Internal representation of the window

   - Instead of adding all the values in the window each time, the sum is updated incrementally: the new value `val` is added and the oldest value, `window(ptr)`, is subtracted. Thus, when a new value is consumed in the `S_RECV` state, `sum` becomes `sum + val - window(ptr)`, `val` is stored in `window(ptr)`, and `ptr` advances to the next position (returning to 0 after the last one). Since signals are updated only when the process is suspended, all these assignments read the *old* values of `sum`, `window(ptr)` and `ptr`, so they are performed in the same clock cycle.
   - The `sum` register is provided directly at the output. It is correct only if it always equals the sum of the values in the window, so the window, the pointer and the sum have to be cleared together on reset.

3. **Connect the previously implemented modules to get a module which computes the sum of a window of values which are pre-saturated to fit in a given range. Create a testbench to simulate your design. The block design of the module is shown in Figure 3.**

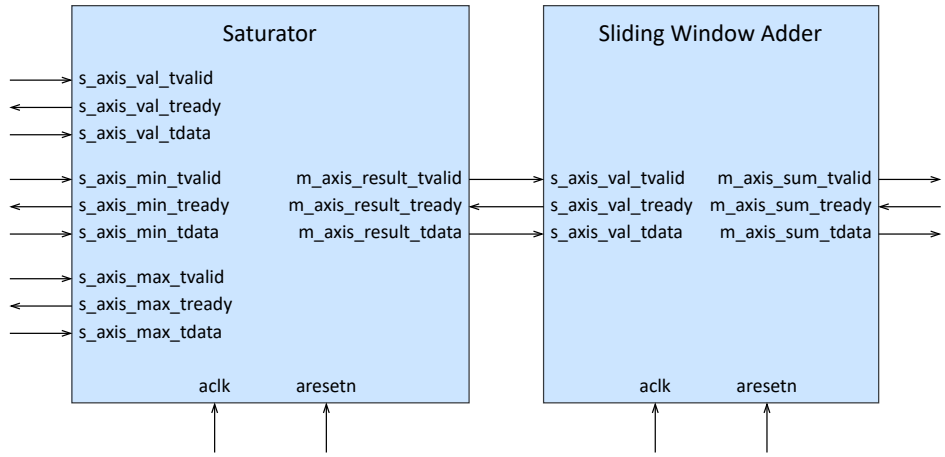


Figure 3: The block design of the module to be implemented

Design and implementation considerations:

- In the testbench, keep `aresetn` LOW for a few clock cycles, then drive the inputs as a correct sender (**Rule 1** and **Rule 2**). Since the saturator consumes `val`, `min` and `max` together, the range limits have to be provided with every value.
- Test the following situations: the three inputs become valid in different clock cycles, and the ready signal of the output interface (`m_axis_sum_tready`) is kept LOW for several clock cycles. Check that no value is lost or consumed twice.
- Example: for `WINDOW_SIZE = 5`, `min = -10`, `max = 10` and the input values $3, 20, -15, 7, 1, 4$, the expected output stream is $3, 13, 3, 10, 11, 12$.

References

[1] AMBA 4 AXI4-Stream Protocol Specification - Arm [accessed Oct. 2026], https://documentation-service.arm.com/static/642583d7314e245d086bc8c9?token=

[2] How the axi-style ready/valid handshake works [accessed Oct. 2026], https://vhdlwhiz.com/how-the-axi-style-ready-valid-handshake-works/